

```
1  #include <stdio.h>
2
3  void bestfit(int mp[], int p[], int m, int n) {
4      int j = 0;
5      for (int i = 0; i < n; i++) {
6          if (mp[i] > p[j]) {
7              printf("\n%d fits in %d", p[j], mp[i]);
8              mp[i] = mp[i] - p[j++];
9              i = i - 1;
10         }
11     }
12     for (int i = j; i < m; i++) {
13         printf("\n%d must wait for its process", p[i]);
14     }
15 }
16
17 void rsort(int a[], int n) {
18     for (int i = 0; i < n; i++) {
19         for (int j = 0; j < n; j++) {
20             if (a[i] > a[j]) {
21                 int t = a[i];
22                 a[i] = a[j];
23                 a[j] = t;
24             }
25         }
26     }
27 }
28
29 void sort(int a[], int n) {
30     for (int i = 0; i < n; i++) {
31         for (int j = 0; j < n; j++) {
32             if (a[i] < a[j]) {
33                 int t = a[i];
34                 a[i] = a[j];
35                 a[j] = t;
36             }
37         }
38     }
39 }
40
41 void firstfit(int mp[], int p[], int m, int n) {
```

Output

```
Number of memory partition : 3
Number of process : 2
Enter the memory partitions :
100
500
200
ENter process size :
200
80
1. Firstfit      2. Bestfit      3. worstfit
Enter your choice : 1

200 fits in 500
80 fits in 300
```

=== Code Execution Successful ===

```

42     sort(mp, n);
43     sort(p, m);
44     bestfit(mp, p, m, n);
45 }
46
47 void worstfit(int mp[], int p[], int m, int n) {
48     rsort(mp, n);
49     sort(p, m);
50     bestfit(mp, p, m, n);
51 }
52
53 int main() {
54     int m, n, mp[20], p[20], ch;
55     printf("Number of memory partition : ");
56     scanf("%d", &n);
57     printf("Number of process : ");
58     scanf("%d", &m);
59     printf("Enter the memory partitions : \n");
60     for (int i = 0; i < n; i++) {
61         scanf("%d", &mp[i]);
62     }
63     printf("Enter process size : \n");
64     for (int i = 0; i < m; i++) {
65         scanf("%d", &p[i]);
66     }
67     printf("1. Firstfit\t2. Bestfit\t3. worstfit\nEnter your choice : ");
68     scanf("%d", &ch);
69     switch (ch) {
70         case 1:
71             bestfit(mp, p, m, n);
72             break;
73         case 2:
74             firstfit(mp, p, m, n);
75             break;
76         case 3:
77             worstfit(mp, p, m, n);
78             break;
79         default:
80             printf("invalid");
81             break;
82     }
83     return 0;
84 }

```